HTML

Context

Templates can access a variety of "contexts" and "variables" to build a document:

- `doc`: Typst's generated HTML shell and document content.
- `site`: website metadata, navigation, and search settings.
- `css`: theme CSS files. Each item has `name` and `content`.
- `js`: theme JS files. Each item has `name` and `content`.
- `highlight_css`: standalone syntax-highlight CSS.
- `theme`: the active theme name.
- `target`: the render target; currently `html`.
- `store`: the completed document store from `[store]`, `calepin.store.set()`, successful `store-set` chunks, and CLI `--set store.<name>=...` overrides.

`doc` contains:

- `doc.head`: Typst's document shell before `</head>`.
- `doc.body_open`: `</head>` and Typst's opening `<body>` tag.
- `doc.body`: Typst's generated body content.
- `doc.body_close`: Typst's closing `</body>` tag and any remaining document shell.
- `doc.title`: the document title, or an empty string.

`site` contains:

- `site.title`, `site.description`, `site.base_url`
- `site.logo`, `site.logo_alt`, `site.home_url`, `site.favicon`
- `site.page_url`, `site.page_title`
- `site.page_pdf`: page-relative link to this page's PDF, when one is published.
- `site.page_source`: whether this page's Typst source is published.
- `site.sidebar`: flat navigation entries.
- `site.sidebar_sections`: grouped navigation sections.
- `site.sidebar_fold`: whether titled sidebar sections should fold.
- `site.toc`: the current page table of contents.
- `site.menus`: named menus, such as `site.menus.main`, `site.menus.social`, and `site.menus.footer`.
- `site.language`: the current language code.
- `site.languages`: language entries for the language picker.
- `site.translations`: alternate-language links for the current page.
- `site.pagefind`: Pagefind search assets and bundle path, when search is enabled.

Nested entries expose these fields:

- Navigation entries in `site.sidebar`, `site.menus.<name>`, and section `items`: `href`, `label`, `label_html`, `active`.
- Sidebar sections: `title`, `active`, `items`.
- TOC entries: `level`, `href`, `label`.
- Language and translation entries: `c de`, `label`, `href`, `active`.
- Pagefind: `css`, `js`, `bundle`.

Layouts

HTML layouts are MiniJinja templates; see Templating for the syntax. For a single HTML notebook, use `layouts/document.html`. For websites, the default entry is `layouts/site.html`.

Here is a minimal `layouts/document.html`:

```
{{ doc.head }}
  <meta charset="UTF-8">
  <title>{{ doc.title }}</title>
  {% for file in css %}
  <style>
{{ file.content }}
  </style>
  {% endfor %}
{{ doc.body_open }}
  <header class="document-header">
    <a href="index.html">Home</a>
    <button type="button" data-calepin-theme-toggle>Theme</button>
  </header>
  <main class="container">
    {{ doc.body }}
  </main>
  {% for file in js %}
  <script>
{{ file.content }}
  </script>
  {% endfor %}
{{ doc.body_close }}
```

Keep `doc.head`, `doc.body_open`, and `doc.body_close` unless you are intentionally replacing the entire HTML shell.

Partials

Instead of maintainng large monolithic templates, it is useful to split templates into different "partials", that is, templates that handle a specific part of a web page. This allows re-useability and makes the codebase more maintainable. These templates are hosted under `partials/`, and we include them in a layout with the `{% include %}` tag.

```
{% include "partials/header.html" %}
```

Partials receive the same template context as the file that includes them. Shared partials from the bundled base layer are included by both built-in themes; a theme-local partial with the same path overrides the shared one. To customize one component, copy that partial into your local theme and edit it there.

The bundled themes currently provide these partials. This list is descriptive; the theme source code on GitHub is canonical.

Shared partials:

- `partials/document-head.html`
- `partials/site-brand.html`
- `partials/site-footer.html`
- `partials/site-head.html`
- `partials/site-head-meta.html`
- `partials/site-language-picker.html`
- `partials/site-nav-prev-next.html`
- `partials/site-search.html` — the Pagefind modal container, not the full search UI.
- `partials/site-sidebar-item.html`

- `partials/theme-init-script.html`
- `partials/theme-scripts.html`
- `partials/theme-styles.html`
- `partials/theme-toggle-button.html`

`calepin` theme partials:

- `partials/document-theme-switcher.html`
- `partials/site-nav.html`
- `partials/site-nav-item.html`
- `partials/site-nav-sidebar.html`
- `partials/site-nav-toc.html`

`academic` theme partials:

- `partials/site-nav.html`
- `partials/site-nav-item.html`

Page-specific layouts

Sometimes, we would like a specific page of a website to use a different layout than the default. For instance, the landing page is often very different than the other pages of a site.

You can switch layouts per page with `<website-metadata>`:

```
#metadata((
  title: "Landing page",
  layout: "layouts/site-landing.html",
)) <website-metadata>
```

The `layout` value must be a relative `.html` path inside the active theme. Calepin does not add `layouts/` or `.html` for you and does not fall back to `layouts/site.html` if the file is missing.